EE-133L P ROGRAMMING FUNDAMENTALS

Laboratory Manual

Student Name: Muhammad Umer Bin Qaiser

Registration No. 2025 – EE – 016

Lab Section: (A)

Department of Electrical Engineering
University of Engineering and Technology, Lahore

Lab Safety and Discipline Rules for Programming Fundamentals Laboratories

1. Students must not work alone in the lab during official hours; they should be supervised by an instructor or lab staff.
2. Always log in to the computer with your assigned credentials. Do not attempt to access unauthorized accounts or systems.
3. Before running your program, carefully check your code, logic, and syntax. Discuss with your partner/classmate if needed and re-check before executing. If unsure, ask your lab instructor.
4. Any modification in your code should be properly saved with version control (different filenames or commits) to avoid accidental data loss.
5. Avoid experimenting with harmful code (e.g., infinite loops, virus scripts, system commands) that may damage the computer or affect other users.
6. After completing the experiment, properly save your work, log out from your account, and shut down/restart the system as instructed.
7. Do not attempt to install or uninstall software on lab computers. If any software issue arises, report it to the lab staff/instructor.
8. Do not use damaged peripherals (mouse, keyboard, power cord, USB) or computers with overheating/faults. Report such issues immediately.
9. When testing or debugging, carefully observe error messages and avoid forceful restarts or shutdowns. If the system hangs, inform the instructor.
10. Use USB drives or cloud storage responsibly. Do not connect unknown/unverified devices to lab computers.
11. Students should wear proper attire in the computer lab. Slippers are not recommended; maintain a professional environment.
12. Do not bring or consume food and beverages inside the computer lab.
13. Avoid working on computers with wet hands or while carrying liquids nearby.
14. Remove loose jewelry, accessories, and keep long hair tied back to prevent interference with fans or moving parts of machines.
15. Always ensure the computer and software you are using are within their allowed capacity (e.g., do not overload the system with unnecessary programs).
16. Never overload the computer by running multiple heavy applications simultaneously.
17. Do not attempt to bypass security settings, firewalls, or antivirus systems.
18. Ensure all external devices (USB, portable drives) are properly scanned and safely removed after use.
19. Backup your code and data regularly to avoid data loss.

INSTRUCTORS:	Ms. Khush Bakht Ms. Bareera Sadia	**SESSION:** 2026	
SUBJECT TITLE:	Programming Fundamentals Lab		

LAB PERFORMANCE EVALUATION RUBRIC

Performance	CLOs Mapping	Excellent (100%–80%)	Good (80%–55%)	Satisfactory (55%–30
Logic / Approach to Problem Solving (30 Points)	**CLO1, CLO2, CLO3.**	Demonstrates clear, innovative, and structured problem-solving approach, selects the most efficient logic and justifies choices effectively.	Applies appropriate logic with minor gaps, solution works with some inefficiency but shows understanding of approach.	Basic attempt to apply log with significant gaps, solu partially correct and lacks
Code Implementation & Error Debugging (40 Points)	**CLO1, CLO2, CLO3.**	Code is well-structured, follows best practices, and compiles successfully on the first attempt; independently debugs errors with precision.	Code is mostly correct with minor issues; requires some debugging but resolves independently.	Code contains multiple err requires assistance of lab instructor for debugging a corrections.
Documentation & Report Writing (20 Points)	**CLO4**	Documentation is comprehensive, professional, and well-formatted, includes clear explanations, comments, and structured reporting.	Documentation is complete with minor formatting or clarity issues; comments are present but limited.	Documentation is minimal clarity, or inconsistent formatting; explanations a insufficient.
Discipline & Lab Ethics (10 Points)	**CLO4**	Arrived on time, fully prepared, and used time well. Maintained clean coding environment (file structure, naming, saving work). Followed all lab policies.	Arrived slightly late (≤5 mins), prepared, used time well, and maintained proper coding environment. Generally followed lab rules.	Arrived late (>5 mins but mins), were partially prepa Completed work but show little interest. Needed remi about coding practices/lab discipline.
TOTAL		<------Total points earned (Max 100) = **Lab Performance Grade**		

List of Experiments

Exp No.	Title
1	Introduction of Ubuntu Setup and Simple C programs.
2	Introduction to Simple C Programs
3	Variables and implementation of Arithmetic Operations in C
4	Introduction to Control Statements
5	Introduction to Loop Structures – I
6	Introduction to Loop Structures – II
	Lab Evaluation 1
	Mid-Semester Exam
7	Introduction to Functions in C
8	Introduction to Recursion in C
9	Single- and Multi-Dimensional Arrays in C
	Lab Evaluation 2
10	Pointers and Dynamic Memory Allocation in C
11	Introduction to Custom Data Types in C
12	File Processing using C
13	Open-Ended Lab – I
14	Open-Ended Lab – II

Experiment no. 1:

Installation of Ubuntu Setup and First C Program

Environment Setup Objectives:

By the end of this lab, students will be able to:

1. Installation of Ubuntu Linux.
2. Basic usage of Linux Terminal.
3. Writing, compiling, and executing a simple C program.
4. Understand the purpose and syntax of the printf function in C.
5. Use escape sequences to control program output (newlines, tabs, quotes, etc.).
6. Format text output using conversion specifiers (e.g., %d, %f, %s).
7. Apply multiple printf statements to produce structured and aligned output.
8. Demonstrate proper commenting and code style as recommended in *P.J. Deitel & H.M. Deitel – C How to Program.*
9. Recognize the role of printf in debugging and user interaction within console applications.

Pre-Lab Requirements:

- Windows 10 or later operating system.
- Administrative privileges on the computer.
- Stable internet connection (minimum 2 Mbps).
- At least 2GB free disk space.
- Basic knowledge of C program structure (#include, main, return 0;).
- Familiarity with using Ubuntu Linux.
- Ability to create and execute a simple C source file (.c).
- Review **Chapter 2 (Introduction to C Programming)** from *Deitel & Deitel* for examples on printf.

Part A: Ubuntu Installation

Step 1: Downloading Ubuntu Setup

- **Ubuntu Desktop Installation Guide (Official):**
  1. **Open** your web browser (Chrome, Firefox, or Edge).
  2. **Navigate** to: https://documentation.ubuntu.com/desktop/en/latest/tutorial/install-ubuntu-desktop/.
  3. **Click** on "Download" and **wait** for the download to complete.
  4. **Locate** the downloaded file (usually in Downloads folder).

- **WSL vs Dual Boot (Comparison Guide):**
  This guide helps students understand the difference between WSL and Dual Boot and decide which option is more suitable. It also provides guidelines for both the installations.
  https://talhaayyaz.medium.com/wsl-vs-dual-boot-which-is-better-1e317 b0cc107

Method 1: Installing Ubuntu on VirtualBox

This method is recommended for beginners and for students who do not want to modify their existing operating system.

Recommended YouTube Tutorial: Installing Ubuntu on VirtualBox
https://www.youtube.com/watch?v=x5MhydijWmc

Method 2: Dual Boot Ubuntu with Windows

This method allows Ubuntu to be installed alongside Windows on the same system.

Recommended YouTube Tutorial: Dual Boot Ubuntu with Windows
https://www.youtube.com/watch?v=mXyN1aJYefc

Important Notes

- ➢ Ensure at least 20 GB of free disk space before installation.
- ➢ Backup important data before proceeding with Dual Boot installation.
- ➢ VirtualBox users must install both VirtualBox and its Extension Pack.

Part B: Introduction to Terminal

After successful installation of Ubuntu:

1. **Log in** to Ubuntu.
2. **Open** the Terminal using:
   - **Ctrl + Alt + T**
   - OR search **Terminal** from the Applications menu.

Part C: Creating Your First C Program

Step 1: Install GCC Compiler

Before running C programs, install the GCC compiler using the following commands:

```
sudo apt update
sudo apt install gcc
```

Verify the installation:

```
gcc -- version
```

Step 2: Creating a Workspace

1. **Create** a new folder on Desktop called "PF_Labs"
2. **Inside PF_Labs**, create folder "Lab01_your_roll_number" e.g. Lab01_25_EE_01.
3. **Create** all your tasks files for one lab in its particular folder.
4. **Create** a new file named myfirstCprogram.c:

```
cd Desktop/PF_labs/Lab01_25_EE_01/
nano myfirstCprogram.c
```

Step 3: Writing Your First Program

1. **Type the following code exactly in editor:**

```
#include <stdio.h>

int main()
{
    printf("My First C Program!\n");
    printf("Welcome to Programming Fundamentals!\n");
    return 0;
}
```

2. **Save** the file with Ctrl + O
3. **Exit** using Ctrl+X

Step 3: Compiling and Running Your Program

Type the following commands and compile using GCC.

```
gcc myfirstCprogram.c -o myfirstCprogram
```

Execute the compiled program:

```
./myfirstCprogram
```

Step 4: Insert Your Output (must having your .c file directory)

After completing your program and ensuring it compiles successfully without errors, capture your program output along with the source file location. This step validates both **code execution** and **directory management skills**.

Instructions:

1. Ensure your C source file is properly saved in the designated lab directory.
   Example directory path: C:\Desktop\PF_Labs\Lab01_25_EE_01\ myfirstCprogram.c

2. Compile your program using **GCC** (or any approved compiler).
   Example command: gcc myfirstCprogram.c -o myfirstCprogram
3. Run the executable and capture the **complete terminal screenshot** showing:

- The compilation command.
- The execution command.
- The program's output.
- The full path of your .c source file.

Insert your code screenshot:

```c
#include <stdio.h>
int main()
{
printf("My First C Program!\n");
printf("Welcome to Programming Fundamentals!\n");
return 0;
}
```

Insert your Output screenshot:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ gcc myfirstCprogram.c -o myfirstCprogram
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ ./myfirstCprogram
My First C Program!
Welcome to Programming Fundamentals!
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$
```

Task 1:

Procedure (step-by-step):

1. Create a file named task1.c.
2. Write a simple program that prints a greeting and demonstrates escapes.
3. Compile and run; fix any syntax errors reported by the compiler.

Sample code:

```c
#include <stdio.h>
int main(void)
{
    printf("Hello, Programming Fundamentals!\n");
    printf("This is my First C program\n");
    return 0;
}
```

Output:

```
[Running] cd "d:\PF\LAb-01\" && gcc task1.c -o task1 && "d:\PF\LAb-01\"task1
Hello, Programming Fundamentals!
This is my First C program

[Done] exited with code=0 in 0.456 seconds
```

Student's Code:

```c
#include <stdio.h>

int main(void){
printf("Greetings from UET-EE!\n");
printf("This is C.\n");

}

```

Student's Output:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ gcc task1.c -o task1
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ ./task1
Greetings from UET-EE!
This is C.
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$
```

Task 2:

Write a code to get your name, roll number and section as the output.

Student's Code:

```c
#include<stdio.h>
int main()
{
printf("My name is Umar\n");
printf("My rollno.is 2025_EE_16\n");
printf("My section is A\n");
return 0;
}
```

Student's Output:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ gcc studentinfo.c -o task2
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ ./task2
My name is Umar
My rollno.is 2025_EE_16
My section is A
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$
```

Introduction to Escape Sequences:

Escape sequences are special character combinations in C that represent actions or characters that are otherwise difficult to express directly. They always begin with a backslash (\).
Common uses:
- Formatting output (new lines, tabs)
- Printing special characters (quotes, backslash, percent sign)
- Controlling text display (backspace, carriage return, alert).

Following table 1 represents some **common C escape sequences** with their meanings and examples.

Table 1: Common C Escape Sequences

Escape Sequence	Meaning	Example Code	Expected Output (ANSI/Linux)	Windows CMD / PowerShell Output
\n	New line	printf("Hello\nWorld");	Hello World	Hello World
\t	Horizontal tab	printf("A\tB\tC");	A B C (tab spacing)	A B C (tab spacing)
\\	Backslash	printf("\\");	\	\
\"	Double quote	printf("\"Hello\"");	"Hello"	"Hello"
\'	Single quote	printf("\'A\'");	'A'	'A'
\?	Question mark	printf("\?");	?	?

	(avoid trigraphs)			
%%	Percent sign	printf("100%%");	100%	100%
\a	Alert (beep)	printf("Beep!\a");	Beep! + system beep	Often just prints Beep! (no sound on most systems)
\b	Backspace	printf("AB\bC");	AC (B erased)	AC (B erased)
\r	Carriage return	printf("Hello\rBye");	Byelo (overwrites Hello)	Byelo (overwrites Hello)
\v	Vertical tab (ASCII 11)	printf("A\vB\n");	A B	A♂B
\f	Form feed (ASCII 12)	printf("Line1\fLine2\n");	Line2 (Line1 cleared or page break)	Line1♀Line2
\0	Null character (string terminator)	printf("Hi\0Hello");	Hi (string ends at \0)	Hi (string ends at \0)

Task 3:

Use multiple printf statements to make hourglass consisting of 7 rows as shown below:

```
*******
*****
***
*
***
*****
*******
```

Student's Code:

```c
#include <stdio.h>

int main(){
printf("      *******\n");
printf("       *****\n");
printf("        ***\n");
printf("         *\n");
printf("        ***\n");
printf("       *****\n");
printf("      *******\n");
}
```

Student's Output:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ gcc hourglass.c -o task3
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ ./task3
      *******
       *****
        ***
         *
        ***
       *****
      *******
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$
```

Task 4:

Following program prints examples of the most common escape sequences. Students should compile and run it, then observe the behavior.

Sample code:

```c
#include <stdio.h>

int main(void)
{
    printf("Hello\nWorld\n");
    printf("A\tB\tC\n");
    printf("\\\n");
    printf("\"Hello\"\n");
    printf("100%%%% complete\n");
    printf("AB\bC\n");
    printf("Hello\rBye\n");
    printf("Beep!\a\n");
    printf("A\vB\n");
    printf("Line1\fLine2\n");
    printf("Hi\0Hello\n");
    return 0;
}
```

Student's Output:

```
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ gcc escape_seq.c -o task4
escape_seq.c: In function 'main':
escape_seq.c:14:11: warning: embedded '\0' in format [-Wformat-contains-nul]
   14 | printf("Hi\0Hello\n");
      |           ^~
muhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$ ./task4
hello
world
A       B       C
\
"Hello"
100%% complete
AC
ByeMonga
Beep
A
 B
Line1
     Line2
Himuhammad-umar@ThinkPad-X395:~/Desktop/PF_Labs/Lab01_2025_EE_016$
```

Extensions: (Bonus marks)

1. Write each task 1, task 2 and task 3 code using single printf statement.
2. Print a table of three columns (Name, Roll No, Marks) using \t for spacing.
3. Write a C program that prints the following using **escape sequences**:
   - A message with a **double quote**, a **backslash**, and a **percent sign**.
   - A word where **characters are erased** using \b.
   - A word overwritten by another using \r.
   - Output that demonstrates **alert (\a)**, **form feed (\f)**, and **vertical tab (\v)**.
   - Properly formatted text using **newline (\n)** and **tab (\t)**.
   - Add comments in your code to explain each escape sequence before it is used.
   - Run the program, compare the actual output with your predictions, and record your observations.
4. Use multiple printf statements to get the output as shown below. Pay special attention to the alignment and position of each text and each hourglass at the output. The integers "1234….0123" will help you to align and position the texts and the hourglasses properly. Also pay attention to the number of rows and the number of asterisks (*) in each row of each hourglass. (**Student who use minimum number of printf statement will be given bonus marks)**.

```
123456789012345678

Hourglass    Hourglass

   1            2

*****
 ***          ***

  *            *

 ***          ***

*****
```